

QUESTION 1

Real-Time Face Recognition for Campus Security

1. Formulating the Problem as a Constraint Satisfaction / Optimization Problem

This system can be modelled as a Constraint Satisfaction Problem (CSP) where a set of design variables must be assigned values (algorithms, thresholds, resolutions) such that every hard constraint from the requirement list is satisfied, while an objective function (accuracy, subject to real-time speed) is optimized.

Variables and Domains:

Variable	Domain (Possible Values)
Face detector algorithm	{Haar Cascade, HOG+SVM, DNN(SSD/ResNet), MTCNN}
Feature representation	{Eigenfaces, Fisherfaces, LBPH, Deep Embedding (128-d)}
Recognition matching rule	{Nearest-Neighbour + threshold, SVM classifier, KNN classifier}
Frame resolution	{160x120, 320x240, 640x480, 1280x720}
Detection frequency	{every frame, every 2nd frame, every 5th frame + tracker}
Match threshold (distance/confidence)	{continuous range, e.g., 0.3–1.0}

Constraints:

- Hard constraint C1 — Real-time: total pipeline latency per frame must stay within the frame interval of the target FPS (e.g., ≤ 66 ms for 15 FPS) on a low-cost CPU.
- Hard constraint C2 — Lighting robustness: the chosen detector/feature combination must remain functional under low-light and backlit conditions.
- Hard constraint C3 — False-recognition minimization: an unregistered face must not be assigned any enrolled identity (i.e., matching rule must include a rejection option).
- Soft constraint (objective) — Maximize recognition accuracy for registered individuals, subject to C1–C3 being satisfied.

2. Solving the CSP — Constraint Propagation and Search

Rather than exhaustively trying every combination of variable values (a brute-force search over the whole domain), the design narrows each variable's domain early using constraint propagation — the CSP idea of eliminating values that cannot satisfy a constraint before search proceeds:

- Applying C1 (real-time) first eliminates heavy, high-accuracy-only options — e.g., large DNN backbones or 1280x720 processing — from the domain of 'detector' and 'resolution', since they cannot meet the latency bound on low-cost hardware.
- Applying C2 (lighting robustness) removes plain grayscale/Eigenface-only approaches from the 'feature representation' domain unless paired with a lighting-normalization pre-step (CLAHE), effectively adding a derived constraint linking two variables together.
- Applying C3 forces the 'matching rule' domain to only include rules with an explicit rejection/threshold mechanism, ruling out a plain 'always return nearest match' classifier.

What remains after this propagation is a much smaller, feasible search space — e.g., {DNN-lite or Haar detector} x {LBPH or deep embedding + CLAHE} x {threshold-based matching} x {reduced resolution + frame-skipping} — from which the final configuration is selected to further maximize accuracy.

3. Resulting Pipeline (Satisfying the Constraint Set)

- Acquisition: cv2.VideoCapture at reduced resolution (satisfies the resolution-domain restriction from C1).
- Preprocessing: CLAHE + Gaussian blur (satisfies C2 by normalizing illumination before detection).
- Detection: Haar cascade (lowest-cost hardware) or a lightweight DNN detector (better lighting/pose robustness) — the choice itself is a constraint-driven trade-off between C1 and C2.
- Feature extraction: deep embedding vector (128-d) or LBPH, chosen for lighting invariance without exceeding the compute budget.
- Recognition: nearest-neighbour distance against enrolled embeddings with a calibrated threshold — this threshold is the tunable parameter that directly encodes constraint C3.
- Optimization: detection run every k frames with a KCF tracker filling the gap — this satisfies C1 without discarding C2/C3, since recognition/detection accuracy is unaffected between detection cycles.

4. Justification — How the Constraints Are Simultaneously Satisfied

Each requirement maps to a specific, checkable condition on a specific variable: latency is bounded by resolution + frame-skipping choices, lighting robustness is bounded by the preprocessing + feature-extraction choice, and false-recognition is bounded by the threshold value. Because these variables are largely decoupled (changing the threshold does not affect FPS, changing resolution does not affect the false-acceptance rate), the constraints can be tuned

independently after the propagation step narrows the feasible space — this is what allows the system to hit real-time performance, lighting robustness, and low false-recognition simultaneously rather than trading one off for another.

QUESTION 2

Intelligent Traffic Violation Detection System

1. Formulating the Problem as a Constraint Satisfaction / Optimization Problem

The task is to assign values to a set of processing-pipeline variables such that the resulting system produces one, and only one, count event per vehicle per crossing, while correctly flagging violations, under noisy real-world conditions.

Variables and Domains:

Variable	Domain (Possible Values)
Foreground extraction method	{Frame differencing, MOG2, KNN, DNN object detector}
Shadow handling	{None, MOG2 shadow flag, HSV-based shadow removal}
Mask cleanup operations	{None, Opening only, Opening+Closing}
Tracking method	{Centroid tracker, Kalman filter + Hungarian (SORT), Correlation tracker}
Counting rule	{Count every frame object is on the line side, Count once per track-ID via flag}
Minimum blob area threshold	{continuous range depending on camera distance/resolution}

Constraints:

- Hard constraint C1 — Each physical vehicle must be counted exactly once (no duplicate or missed counts).
- Hard constraint C2 — Shadows, camera noise, and swaying background elements must not be counted as vehicles.
- Hard constraint C3 — The system must operate in real time on the incoming video stream.
- Soft constraint (objective) — Maximize correct detection of abnormal/violating movements (wrong-direction, speeding, signal violation).

2. Solving the CSP — Propagation and Elimination of Invalid Configurations

- C1 directly eliminates 'counting rule = count every frame on the line side' from the domain, since this would generate many counts per vehicle while it lingers near the line — leaving only the track-ID-flag-based counting rule as feasible.
- Because a track-ID-based counting rule requires a persistent identity, this constraint propagates backwards and forces the 'tracking method' domain to exclude simple frame-independent detection (no tracker at all), narrowing it to centroid or Kalman-based tracking.
- C2 removes 'shadow handling = None' from the domain, since shadows would otherwise inflate the foreground mask into false vehicle-shaped blobs; it also pushes the minimum-area/aspect-ratio threshold to a non-trivial (non-zero) value.
- C3 restricts 'foreground extraction method' away from heavy full-frame DNN detection at every frame (unless a lighter tiny-YOLO/MobileNet-SSD variant is used), favouring MOG2/KNN background subtraction as the primary low-cost option, with detection used periodically to re-anchor tracks.

After this elimination, the feasible configuration space converges to: MOG2 with shadow detection, morphological opening+closing, minimum-area filtering, and Kalman/centroid tracking with per-track counting flags — which is then used as the operating pipeline.

3. Resulting Pipeline (Satisfying the Constraint Set)

- Preprocessing: grayscale + Gaussian blur + ROI cropping to the road area.
- Foreground extraction: MOG2/KNN with detectShadows=True — directly satisfying the shadow sub-constraint of C2.
- Mask cleanup: opening (noise removal) then closing (blob consolidation) — satisfying the remainder of C2 against camera noise.
- Tracking: Kalman-filter-based tracker assigning a persistent ID per vehicle — the structural requirement created by C1.
- Counting: a boolean 'counted' flag stored per track ID, set the instant the centroid crosses the virtual line — the direct encoding of C1.
- Violation logic: direction vector and calibrated speed estimate computed from the same tracked trajectory, satisfying the objective without needing additional heavy computation (reusing C1's tracking output).

4. Justification

Framing the requirements as constraints on specific variables (counting rule, shadow handling, tracker type) makes clear why naive designs fail: without a persistent track ID, C1 is structurally unsatisfiable regardless of how good the detector is; without explicit shadow handling, C2 is violated regardless of tracking quality. Solving the CSP by propagating constraints before choosing a final configuration is what leads directly to a design combining adaptive background subtraction, shadow suppression, and ID-persistent tracking — rather than arriving at it by trial and error.

QUESTION 3

Automated Document Processing System

1. Formulating the Problem as a Constraint Satisfaction / Optimization Problem

Each scanned document must be transformed by a sequence of operations (variables) chosen from valid technique-sets (domains) such that the final image satisfies the input requirements of an OCR engine, across a batch of documents with heterogeneous defects — without any manual per-file tuning.

Variables and Domains:

Variable	Domain (Possible Values)
Noise-removal filter	{Gaussian blur, Median blur, Non-Local Means Denoising}
Binarization method	{Global Otsu, Adaptive Mean, Adaptive Gaussian}
Morphological repair ops	{None, Erosion/Dilation, Opening+Closing}
Skew-correction method	{None, Hough-line based, minAreaRect based}
Segmentation strategy	{Whole-page OCR, Contour-based region OCR, Projection-profile line OCR}

Constraints:

- Hard constraint C1 — Output image must be near-binary, high-contrast (black text / white background) regardless of input illumination.
- Hard constraint C2 — Text lines must be horizontal (skew within OCR's tolerance, typically a few degrees) before OCR is applied.
- Hard constraint C3 — The pipeline must apply uniformly across a batch without file-specific parameter tuning (full automation).
- Soft constraint (objective) — Maximize OCR text-extraction accuracy.

2. Solving the CSP — Propagation Across the Pipeline

- C1 eliminates 'Global Otsu' from the binarization domain whenever illumination is known to be uneven (the common case for scanned batches), leaving Adaptive Mean/Gaussian thresholding as the feasible values — since only adaptive thresholding can satisfy C1 without per-file brightness tuning (which C3 forbids).

- C3 (no manual tuning) simultaneously restricts the 'noise-removal filter' domain away from parameter-heavy, hand-tuned filters, favouring Median blur or Non-Local Means Denoising with fixed default parameters that generalize across documents.
- C2 makes 'skew-correction method = None' infeasible whenever any scanned or photographed input could arrive rotated, propagating a requirement for an automatic angle-estimation step (Hough-line or minAreaRect) rather than assuming pages arrive perfectly aligned.
- The objective (maximize OCR accuracy) then selects, among the remaining feasible values, the segmentation strategy that best handles the stated mixed font sizes — ruling out 'whole-page OCR' in favour of contour/projection-based region segmentation.

3. Resulting Pipeline (Satisfying the Constraint Set)

- Grayscale conversion + Median blur / Non-Local Means denoising — satisfies C3 by using fixed, non-file-specific parameters.
- CLAHE / illumination normalization prior to thresholding — a supporting step that makes adaptive thresholding reliable.
- Adaptive thresholding (Gaussian) — the direct satisfier of C1.
- Morphological opening+closing — repairs stroke breaks left by binarization, protecting OCR accuracy.
- Automatic skew-angle detection and rotation — the direct satisfier of C2.
- Contour/projection-based segmentation into text regions before OCR — chosen to maximize the accuracy objective given mixed font sizes and multi-column layouts.

4. Justification

Each requirement in the brief corresponds to exactly one constraint on exactly one pipeline variable, and satisfying C3 (automation) is what forces adaptive, parameter-fixed techniques to be chosen over classical but tuning-dependent ones (e.g., adaptive thresholding over manually-set global thresholds). This constraint-first view is why the pipeline design is not arbitrary — it is the smallest set of automated, general-purpose operations that jointly satisfy binarization quality, alignment, and batch-scale automation.

QUESTION 4

Real-Time Multi-Person Tracking and Attendance System

1. Formulating the Problem as a Constraint Satisfaction / Optimization Problem

The system must assign an attendance state (Present/Not-yet-seen) to every enrolled student such that each present student is marked exactly once per session, identity is preserved across frames despite motion/occlusion, and processing delay stays low — a constraint problem over both perception variables (detection/recognition) and bookkeeping variables (tracking/state).

Variables and Domains:

Variable	Domain (Possible Values)
Multi-face detector	{Haar (multi-scale), DNN-SSD, MTCNN}
Recognition invocation policy	{Every frame for every face, Only for new/unconfirmed tracks}
Tracking method	{None (per-frame only), Centroid tracker, Kalman-filter SORT}
Attendance-marking granularity	{Per-frame mark, Per-track mark, Per-student-flag mark}
Track-loss tolerance (missed frames before deletion)	{0, small (~5), moderate (~30), large (~100)}

Constraints:

- Hard constraint C1 — A student must be marked present exactly once per session (no duplicates).
- Hard constraint C2 — Identity must persist through brief occlusion or head movement (no false 're-appearance' as a new person).
- Hard constraint C3 — Processing delay per frame must remain low enough for continuous real-time operation with multiple faces.
- Soft constraint (objective) — Maximize correct-recognition rate.

2. Solving the CSP — Propagation Across Variables

- C1 eliminates 'attendance-marking granularity = per-frame mark' and 'per-track mark' from the domain (both would create duplicates if a track is lost and re-created, or across multiple frames), leaving only 'per-student-flag mark' as feasible.

- C2 forces 'track-loss tolerance = 0' out of the domain (an intolerant tracker would fragment identity on every minor occlusion), requiring a moderate tolerance value combined with motion prediction — which in turn requires 'tracking method' to be Kalman-filter based rather than none.
- C3 eliminates 'recognition invocation policy = every frame for every face' from the domain once more than a few faces are present simultaneously (the compute cost scales with face count x frame rate), leaving 'only for new/unconfirmed tracks' as the feasible value — recognition is expensive, tracking is cheap, so the constraint pushes work from the expensive variable onto the cheap one.
- These three propagated restrictions are mutually compatible — persistent tracking (needed for C2) is exactly what makes per-track-then-per-student marking (needed for C1) and reduced recognition frequency (needed for C3) possible — so the CSP converges to a single coherent configuration rather than requiring further trade-off search.

3. Resulting Pipeline (Satisfying the Constraint Set)

- DNN multi-face detector per frame to locate all students simultaneously.
- Kalman-filter multi-object tracker (SORT-style) assigning a persistent track ID to each physical person — satisfies C2's identity-persistence requirement via motion prediction through short gaps.
- Recognition (embedding + nearest-neighbour match) invoked only when a track is new/unconfirmed, then bound permanently to that track ID — satisfies C3 by not repeating the expensive step.
- A per-student 'already marked' database flag, set once on first confident recognition and never reset during the session — the direct satisfier of C1.
- A moderate missed-frame tolerance before a track is deleted, balancing C2 (don't lose identity on a head-turn) against not indefinitely keeping stale tracks alive.

4. Justification

Because duplicate-prevention (C1) is only achievable once identity is stable (C2), and identity stability is only affordable in real time if recognition is decoupled from tracking (C3), the three constraints are hierarchically linked rather than independent — satisfying C2 first is a prerequisite for satisfying C1 and C3, which is why the pipeline is built tracking-first, with recognition and attendance-marking layered on top of a stable track ID rather than run independently every frame.

QUESTION 5

Smart Home Surveillance and Intrusion Detection

1. Formulating the Problem as a Constraint Satisfaction / Optimization Problem

The system must classify each moving region in the frame as either 'nuisance' (shadow, noise, foliage) or 'genuine object', and raise an alert only when a genuine object satisfies a zone/dwell condition — a labelling/assignment problem constrained by the need to reject several specific classes of false motion while still detecting real intrusions in real time.

Variables and Domains:

Variable	Domain (Possible Values)
Background model	{Static frame differencing, MOG2, KNN}
Shadow suppression	{None, MOG2 shadow flag, HSV-ratio shadow test}
Background learning rate	{Fixed/high (fast-adapting), Low (slow-adapting), Adaptive}
Candidate filter	{None, Minimum-area filter, Area + aspect-ratio filter}
Motion confirmation method	{Single-frame contour only, Multi-frame optical-flow consistency}
Alert trigger rule	{Immediate on detection, Dwell-time threshold inside zone}

Constraints:

- Hard constraint C1 — Shadows must not trigger a false alert.
- Hard constraint C2 — Gradual illumination change and small dynamic-background motion (foliage) must not trigger a false alert.
- Hard constraint C3 — A genuine person entering the restricted zone must be detected and alerted.
- Soft constraint — Operate within the compute budget of limited home-surveillance hardware (favour lighter-weight domain values when multiple satisfy C1–C3).

2. Solving the CSP — Propagation Across Variables

- C1 eliminates 'shadow suppression = None' from the domain outright, since any background-subtraction method without shadow handling will classify a moving shadow as foreground; this leaves MOG2's shadow flag (computationally cheapest) as the preferred feasible value under the soft compute constraint.

- C2 eliminates 'background learning rate = Fixed/high' (which would treat legitimate scene drift as transient, causing model instability) in favour of a slow/adaptive rate, and simultaneously requires 'candidate filter' to include an aspect-ratio/size check so that leaf-sized blobs are excluded regardless of the background model's sensitivity.
- C2 also motivates upgrading 'motion confirmation method' from single-frame contour checking to multi-frame optical-flow consistency, since foliage produces motion but not consistent directional motion — a distinction only a multi-frame check can make.
- C3 eliminates 'alert trigger rule = Immediate on detection' in the presence of the other constraints, since immediate triggering on any surviving candidate would still occasionally fire on transient noise; a dwell-time threshold is required so that only a sustained, zone-confined presence escalates to an alert, without weakening sensitivity to an actual intruder who (unlike noise) remains in the zone for more than a few frames.

3. Resulting Pipeline (Satisfying the Constraint Set)

- MOG2/KNN background subtraction with shadow detection enabled and a slow adaptive learning rate — jointly satisfies C1 and half of C2.
- Morphological opening+closing and a minimum-area/aspect-ratio filter — completes the C2 requirement by removing small nuisance blobs.
- Multi-frame optical-flow direction consistency check — the confirmatory step distinguishing real motion from foliage/reflection flicker.
- Object tracking (centroid/KCF) plus a polygon zone test with a dwell-time threshold before alerting — the direct satisfier of C3 without reopening C1/C2.

4. Justification

The three hard constraints target three distinct nuisance sources (shadows, gradual/small background motion, and premature/noisy alerts), and because each is satisfied by a different, largely independent variable (shadow flag, learning rate + filtering, dwell-time rule), they can all be satisfied concurrently rather than traded off — which is why the final design layers shadow suppression, adaptive background modelling, and dwell-time alerting rather than relying on any single technique to do all the filtering.